

AI ENGINEER & AI PRODUCT DEVELOPER

Complete Interview Preparation Guide

From Fundamentals to System Design | ML to LLMs | Technical to HR

Python & Math	Machine Learning	Deep Learning	NLP & LLMs
MLOps & Design	Gen AI	AI Product	HR Round

Crack Your Dream AI Role — One Question at a Time

How to Use This Guide

This guide is your all-in-one companion for cracking interviews at top AI-first companies — from startups to FAANG. It covers **every topic you will encounter** across screening calls, technical rounds, system design interviews, and final HR discussions. Each section includes concept summaries, real interview questions with difficulty ratings, and answer hints. Study in order or jump to what you need most.

- Questions rated: ● Easy ●● Medium ●●● Hard — focus on Medium/Hard for top companies.
- Each section starts with core concepts, then progresses to interview questions.
- The AI Product Developer sections are clearly marked — skip if applying only for engineering.
- HR section at the end includes the STAR method framework with sample answers.
- Recommended study plan: 4-6 weeks for freshers, 2-3 weeks for experienced professionals.

Typical AI Interview Process

Round	Format	What's Tested	Duration
1. Resume Screen	HR / Recruiter	Keywords, projects, experience match	15–20 min
2. Online Assessment	Coding platform	Python, DSA, ML theory MCQs	60–90 min
3. Technical Round 1	Video call	ML/DL fundamentals, coding	45–60 min
4. Technical Round 2	Video call	Advanced ML, NLP, system design	60 min
5. ML System Design	Whiteboard/Video	End-to-end ML system architecture	45–60 min
6. AI Product Round*	Video call	Product thinking, metrics, roadmap	45 min
7. HR / Culture Fit	Video/In-person	Behavioural, motivation, salary	30–45 min

* AI Product Developer role only. Engineers may skip Round 6.

SECTION 1 — Python & Programming Fundamentals

Python is the lingua franca of AI. Interviewers will test your ability to write clean, efficient code — not just theory. Focus on data structures, OOP, and Python-specific idioms used heavily in AI workflows.

Topic	Key Things to Know
Data Structures	Lists, dicts, sets, tuples, heaps, queues — time complexities
OOP in Python	Classes, inheritance, dunder methods (<code>__repr__</code> , <code>__len__</code> , <code>__iter__</code>)
Functional Python	Lambda, map, filter, reduce, list/dict comprehensions, generators
NumPy Essentials	Broadcasting, vectorisation, axis operations, array slicing
Pandas Essentials	DataFrame ops, groupby, merge, apply, handling NaN
Complexity	Big-O for common operations — crucial for coding rounds
Multithreading/async	GIL, asyncio, concurrent.futures for ML pipelines

Interview Questions — Python & Coding

Q: Explain Python's GIL and how it affects multi-threaded ML code.

●● Medium

Hint: GIL prevents true parallelism in CPython threads; use multiprocessing or C extensions (NumPy) to bypass it.

Q: How does NumPy broadcasting work? Give an example.

●● Medium

Hint: Broadcasting expands arrays with size-1 dimensions to match shapes — avoids explicit loops.

Q: Write a generator function to stream a large dataset in batches.

● Easy

Hint: Use `yield` inside a function with a `batch\_size` parameter; enables memory-efficient data loading.

Q: What is the difference between `@staticmethod` and `@classmethod` in Python?

● Easy

Hint: `@staticmethod` has no implicit first arg. `@classmethod` receives `cls` as first arg — used for factory methods.

Q: Implement a custom iterator for a dataset class in PyTorch style.

●●● Hard

Hint: Implement `__iter__` and `__next__` (or `__getitem__` and `__len__` for map-style datasets).

Q: How would you profile and optimise a slow Python ML training loop?

●●● Hard

Hint: Use `cProfile`, `line_profiler`; vectorise operations; use `DataLoader` with `num_workers`; check GPU utilisation.

SECTION 2 — Mathematics & Statistics for AI

Strong mathematical foundations separate good AI engineers from great ones. You don't need a PhD, but you must understand the 'why' behind algorithms — linear algebra, calculus (for backprop), probability, and statistics are non-negotiable.

Area	Topics You Must Know
Linear Algebra	Vectors, matrices, dot product, eigenvalues, SVD, PCA derivation
Calculus	Derivatives, chain rule, partial derivatives, Jacobian, gradient descent
Probability	Bayes theorem, distributions (Gaussian, Bernoulli, Poisson), MLE, MAP
Statistics	Mean/variance/std, hypothesis testing, p-value, confidence intervals
Information Theory	Entropy, KL divergence, cross-entropy loss, mutual information
Optimisation	Convexity, saddle points, Lagrange multipliers, constrained optimisation

Interview Questions — Maths & Statistics

Q: Explain the intuition behind eigenvalues and eigenvectors. Where are they used in ML?

●● Medium

Hint: Eigenvectors are directions unchanged by a transformation; eigenvalues are scaling factors. Used in PCA, spectral clustering.

Q: Derive the gradient of cross-entropy loss with respect to softmax output.

●●● Hard

Hint: Result simplifies to (predicted - actual) per class due to softmax-log cancellation — a clean derivation to memorise.

Q: What is the difference between MLE and MAP estimation?

●● Medium

*Hint: MLE maximises likelihood $P(\text{data}|\text{params})$. MAP adds a prior: maximises $P(\text{params}|\text{data})$ proportional to likelihood * prior.*

Q: Explain KL divergence. Is it symmetric? When is it used in deep learning?

●●● Hard

Hint: $KL(P||Q)$ measures how much Q diverges from P . Not symmetric. Used in VAEs, knowledge distillation, policy gradients.

Q: What does it mean for a function to be convex? Why does it matter for optimisation?

● Easy

Hint: A convex function has a single global minimum. Guarantees gradient descent converges — unlike non-convex deep nets.

Q: Explain Bayes theorem with a real-world ML example.

● Easy

*Hint: $P(\text{class}|\text{features})$ proportional to $P(\text{features}|\text{class}) * P(\text{class})$. Foundation for Naive Bayes classifiers and Bayesian networks.*

SECTION 3 — Machine Learning Concepts

The backbone of every AI interview. You must be able to explain algorithms intuitively, discuss their trade-offs, and know when to apply which technique. Go beyond sklearn — understand the math inside.

Algorithm	Type	Key Hyperparameters	Common Use Cases
Linear / Logistic Regression	Supervised	Regularisation (L1/L2), learning rate	Baselines, interpretable models
Decision Trees	Supervised	max_depth, min_samples_split	Feature importance, interpretability
Random Forest	Ensemble	n_estimators, max_features	Tabular data, feature selection
Gradient Boosting (XGBoost)	Ensemble	learning_rate, n_estimators, depth	Competitions, structured data
SVM	Supervised	C, kernel, gamma	High-dim data, small datasets
K-Means / DBSCAN	Unsupervised	k (clusters), epsilon, min_samples	Segmentation, anomaly detection
KNN	Lazy learning	k, distance metric	Recommendation, classification
Naive Bayes	Probabilistic	smoothing (alpha)	Text classification, spam detection
PCA / t-SNE / UMAP	Dim. Reduction	n_components, perplexity	Visualisation, compression

Core Concepts to Master

- Bias-Variance Tradeoff: High bias = underfitting; High variance = overfitting. Regularisation balances both.
- Cross-Validation: k-fold, stratified k-fold, time-series CV — always use for model selection.
- Evaluation Metrics: Accuracy, Precision, Recall, F1, ROC-AUC, PR-AUC, RMSE, MAPE — know when to use each.
- Feature Engineering: One-hot encoding, target encoding, feature scaling (Standard vs MinMax vs Robust).
- Imbalanced Data: SMOTE, class weights, threshold tuning, undersampling — critical for real-world problems.
- Regularisation: L1 (Lasso — sparsity), L2 (Ridge — small weights), ElasticNet (both), Dropout (neural nets).

Interview Questions — Machine Learning

Q: Explain the Bias-Variance trade-off with an example.

● Easy

Hint: A polynomial of degree 1 underfit (high bias). Degree 20 overfits (high variance). Degree ~3-5 is sweet spot for many datasets.

Q: When would you use Random Forest vs XGBoost?

●● Medium

Hint: RF is faster to tune and robust to outliers. XGBoost usually wins on structured data with tuning. Start with RF baseline.

Q: How do you handle class imbalance in a fraud detection dataset (1:1000)?

●● Medium

Hint: Use SMOTE, adjust class_weight, use PR-AUC (not accuracy), tune decision threshold on validation set.

Q: Explain the difference between bagging and boosting.

● Easy

Hint: Bagging: parallel independent trees, reduces variance (Random Forest). Boosting: sequential corrective trees, reduces bias (XGBoost).

Q: How would you detect and handle data leakage in a pipeline?

●●● Hard

Hint: Leakage if validation score >> production. Check feature creation timestamps, apply all transforms inside cross-val fold.

Q: Explain how Gradient Boosting builds trees. What is the loss function being minimised?

●●● Hard

Hint: Each tree fits residuals (negative gradient of loss). For regression: MSE; classification: log-loss. Additive sequential ensemble.

Q: What is the curse of dimensionality and how does it affect ML models?

●● Medium

Hint: As features increase, data becomes sparse; distances become meaningless. Feature selection, PCA, or regularisation mitigates it.

Q: How do you choose the right evaluation metric for a classification problem?

●● Medium

Hint: If class imbalance: F1, PR-AUC. Medical: maximise Recall. Spam: maximise Precision. Multi-class: macro/weighted F1.

SECTION 4 — Deep Learning & Neural Networks

Deep learning is at the heart of modern AI. You need solid understanding of architectures, training dynamics, optimisers, and debugging techniques. For senior roles, expect architectural design questions.

Architecture	Best For	Key Innovation
Feedforward (MLP)	Tabular data, regression	Universal approximator
CNN	Images, spatial data	Local receptive fields, weight sharing
RNN / LSTM / GRU	Sequences, time series	Hidden state carries memory
Transformer	NLP, vision, multimodal	Self-attention, positional encoding
Autoencoder	Compression, anomaly detection	Encoder-decoder bottleneck
VAE	Generative, latent space	Reparameterisation trick + KL divergence
GAN	Image synthesis, data augmentation	Adversarial min-max game
Diffusion Models	High-quality image generation	Denoising score matching
Graph Neural Net	Graph-structured data	Message passing between nodes

Key Training Concepts

- Backpropagation: Chain rule applied backward through computation graph — gradient flows from loss to weights.
- Activation Functions: ReLU (fast, dying ReLU issue), GELU (transformers), Sigmoid (output), Tanh, Swish.
- Optimisers: SGD + momentum (robust), Adam (fast convergence), AdamW (weight decay fixed), Lion (memory efficient).
- Batch Normalisation vs Layer Norm: BatchNorm normalises across batch (CNNs). LayerNorm across features (Transformers).
- Dropout: Randomly zeroes neurons during training — regularisation, not inference (use `model.eval()`).
- Learning Rate Scheduling: Warmup + cosine decay is standard for large model training today.
- Gradient Clipping: Prevents exploding gradients in RNNs/Transformers — clip by norm, threshold ~1.0.
- Mixed Precision Training: FP16/BF16 for forward pass, FP32 master weights — 2x speedup, less memory.

Interview Questions — Deep Learning

Q: Explain the vanishing gradient problem. How do ResNets solve it?

●● Medium

Hint: Gradients shrink exponentially through deep layers. ResNets add skip connections: gradient highways bypass layers.

Q: What is the attention mechanism? Derive scaled dot-product attention.

●●● Hard

*Hint: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$. Scaling prevents vanishing softmax gradients in high dimensions.*

Q: Compare BatchNorm and LayerNorm. When would you use each?

●● Medium

Hint: BatchNorm: across batch dimension, great for CNNs. LayerNorm: across feature dimension, essential for Transformers.

Q: How does dropout work during training vs inference?

● Easy

Hint: Training: randomly zero activations with probability p . Inference: disable dropout, scale by $(1-p)$ or just `model.eval()`.

Q: Why does Adam sometimes generalise worse than SGD?

●●● Hard

Hint: Adam adapts per-parameter LR — can overfit to training set. SGD with momentum often finds flatter minima that generalise better.

Q: What is the difference between a VAE and a GAN?

●● Medium

Hint: VAE: encoder-decoder with variational latent space, stable training, blurry samples. GAN: adversarial, sharper samples, harder to train.

Q: Explain how gradient clipping works and when you'd use it.

● Easy

Hint: If gradient norm $>$ threshold, scale gradients proportionally. Use in RNNs, Transformers, and any deep network prone to explosion.

Q: How would you debug a neural network that isn't learning?

●●● Hard

Hint: Check: loss not decreasing (LR too low/high, dead ReLUs), NaN loss (exploding grads), overfit tiny batch first, check data pipeline.

SECTION 5 — NLP, Large Language Models & Generative AI

NLP and LLMs are the hottest areas in AI right now. Every AI company expects you to understand transformers deeply, know how LLMs are built and fine-tuned, and be aware of the latest developments in GenAI.

Concept	What to Know
Tokenisation	BPE, WordPiece, SentencePiece — subword tokenisation, vocabulary size trade-offs
Word Embeddings	Word2Vec (skip-gram, CBOW), GloVe, FastText, contextual embeddings (BERT)
Transformer Architecture	Multi-head self-attention, positional encoding, encoder-decoder, feed-forward layers
BERT Family	Masked LM pre-training, fine-tuning, [CLS] token, sentence-pair tasks
GPT Family	Causal LM (next token prediction), autoregressive generation, scaling laws
Fine-Tuning Strategies	Full fine-tune, LoRA, QLoRA, adapter layers, prefix tuning, prompt tuning
RLHF	Supervised fine-tune → Reward model → PPO — how ChatGPT-style alignment works
RAG	Retrieval-Augmented Generation: embed docs → retrieve → augment context → generate
Prompt Engineering	Zero-shot, few-shot, chain-of-thought, tree-of-thought, role prompting
Evaluation	BLEU, ROUGE, BERTScore, perplexity, human eval, LLM-as-judge

LLM Ecosystem — Tools & Frameworks to Know

- Hugging Face Transformers & Datasets — industry standard for model loading, fine-tuning, and evaluation.
- LangChain / LlamaIndex — for building RAG pipelines, agents, and multi-step LLM applications.
- PEFT / LoRA — parameter-efficient fine-tuning; LoRA adds low-rank matrices to frozen weights.
- vLLM / TGI — high-throughput LLM inference servers with PagedAttention and continuous batching.
- Vector Databases — Pinecone, Weaviate, Chroma, Qdrant — for embedding storage and semantic search.
- Weights & Biases / MLflow — experiment tracking for LLM fine-tuning runs.
- OpenAI API / Anthropic API — for building on top of foundation models with prompt engineering.

Interview Questions — NLP & LLMs

Q: Explain the Transformer architecture from scratch.

●●● Hard

Hint: Encoder: multi-head self-attention + FFN + residuals + LayerNorm. Positional encoding adds sequence order. Decoder adds cross-attention.

Q: What is the difference between BERT and GPT? When to use each?

● Easy

Hint: BERT: bidirectional encoder, good for classification/NER/QA. GPT: causal decoder, good for generation. Both use transformers.

Q: Explain LoRA fine-tuning. Why is it efficient?

●●● Hard

Hint: Adds trainable low-rank matrices A and B alongside frozen weights: $W = W_0 + AB$. Only A, B trained. Reduces params by 100-1000x.

Q: What is RAG and when would you use it over fine-tuning?

●● Medium

Hint: RAG retrieves relevant docs at inference to augment context. Use when knowledge is dynamic/proprietary. Fine-tune for style/format changes.

Q: How does RLHF work? What are its limitations?

●●● Hard

Hint: SFT → reward model from human preferences → PPO to maximise reward. Limitations: reward hacking, expensive labelling, distributional shift.

Q: What are hallucinations in LLMs? How do you mitigate them?

●● Medium

Hint: LLMs generate plausible-sounding but false content. Mitigate: RAG, grounding to docs, confidence thresholds, citations, self-consistency.

Q: Explain positional encoding in Transformers. Why can't we just use index integers?

●● Medium

Hint: Integers don't encode relative positions or extrapolate to longer sequences. Sinusoidal PE encodes absolute + relative position info.

Q: How do you evaluate the quality of an LLM-generated output?

●● Medium

Hint: Automatic: BLEU (translation), ROUGE (summarisation), BERTScore (semantic). Human: accuracy, fluency. LLM-as-judge is emerging standard.

Q: What is chain-of-thought prompting? When does it help?

● Easy

Hint: Adding 'Let's think step by step' elicits reasoning traces before the final answer. Helps for multi-step maths and logic, not simple lookups.

SECTION 6 — MLOps, System Design & Infrastructure

Senior AI engineers and applied scientists are expected to design scalable, reliable ML systems. MLOps is now a core competency — from data pipelines to model monitoring to CI/CD for ML.

Category	Tools / Technologies
Experiment Tracking	MLflow, Weights & Biases, Neptune, Comet ML
Model Registry	MLflow Model Registry, SageMaker Model Registry, Hugging Face Hub
Feature Stores	Feast, Tecton, Hopworks — serve + store features consistently
Orchestration	Apache Airflow, Prefect, Kubeflow Pipelines, Metaflow, ZenML
Model Serving	FastAPI, TorchServe, Triton, Ray Serve, BentoML, SageMaker Endpoints
Containerisation	Docker, Kubernetes — reproducible environments, scaling
CI/CD for ML	GitHub Actions, Jenkins — automated testing, model validation gates
Monitoring	Evidently AI, Arize, Fiddler — data drift, model drift, performance
Cloud Platforms	AWS SageMaker, GCP Vertex AI, Azure ML — managed ML infrastructure

ML System Design Framework (Use in Interviews)

- Step 1 — Clarify Requirements: Scope (online/batch), latency SLAs, data volume, accuracy targets.
- Step 2 — Data: Sources, collection, labelling strategy, ETL pipeline, feature engineering.
- Step 3 — Model Selection: Baseline → simple model → complex model. Justify trade-offs.
- Step 4 — Training Infrastructure: Compute (GPU/TPU), distributed training, experiment tracking.
- Step 5 — Evaluation: Offline metrics, online A/B testing, guard-rails, human evaluation.
- Step 6 — Serving: Latency vs throughput, batching, caching, model compression (quantisation/distillation).
- Step 7 — Monitoring: Data drift, concept drift, prediction distribution, feedback loops.
- Step 8 — Iteration: Continuous training, shadow deployment, champion-challenger, canary releases.

Interview Questions — MLOps & System Design

Q: Design a real-time recommendation system for an e-commerce platform.

●●● Hard

Hint: Two-stage: candidate retrieval (ANN on embeddings) + ranking model. Feature store for user history. A/B test. Monitor CTR.

Q: How do you detect and handle model drift in production?

●● Medium

Hint: Monitor input feature distributions (PSI, KS test) and output distributions. Trigger retraining on drift alerts.

Q: Explain the difference between online and batch inference. When to use each?

● Easy

Hint: Batch: pre-compute predictions offline (low latency requirement). Online: real-time per-request (personalisation, fraud).

Q: How would you reduce ML model inference latency from 200ms to under 20ms?

●●● Hard

Hint: Quantisation (INT8), pruning, knowledge distillation, TensorRT, ONNX, caching frequent queries, smaller architecture.

Q: What is a feature store and why is it important?

●● Medium

Hint: Centralised repository to compute, store, and serve features consistently for training and inference — prevents training-serving skew.

Q: How do you implement a CI/CD pipeline for an ML model?

●●● Hard

Hint: Code commit → unit tests → data validation → model training → evaluation gate (if metric > threshold) → staging → production.

SECTION 7 — AI Product Developer (Product Thinking)

AI Product Developers bridge technical AI capabilities with business impact. You need to think like a PM with deep AI knowledge — define problems, set metrics, build roadmaps, and communicate with both engineers and stakeholders.

Framework	Description	When to Use
CIRCLES Method	Comprehend, Identify, Report, Cut, List, Evaluate, Summarise	Product design questions
RICE Scoring	Reach, Impact, Confidence, Effort — feature prioritisation score	Roadmap prioritisation
HEART Framework	Happiness, Engagement, Adoption, Retention, Task Success	Defining success metrics
Jobs To Be Done	What 'job' does the user hire your product to do?	User research & positioning
North Star Metric	Single metric that captures core product value	Aligning team focus
OKRs	Objective + Key Results — measurable goals at company/team level	Goal setting & tracking

Key AI Product Concepts

- Defining AI Metrics: Choose leading indicators (engagement, latency) AND lagging indicators (revenue, churn).
- Build vs Buy vs Partner: Evaluate AI capabilities — own differentiated models, buy commoditised AI APIs.
- Ethical AI: Bias in training data, fairness metrics (demographic parity, equalised odds), explainability (LIME, SHAP).
- Feedback Loops: Positive feedback (popularity bias in recommendations) vs negative (user corrections improving model).
- Responsible AI: Data privacy (GDPR), model cards, transparency, consent, audit trails.
- Technical Debt in AI: Retraining costs, pipeline complexity, model versioning, feature deprecation.

Interview Questions — AI Product Development

Q: How would you measure the success of an AI-powered search feature?

●● Medium

Hint: Metrics: search success rate, zero-result rate, time-to-click, relevance rating (5-star), DAU using search, revenue influenced.

Q: Design an AI feature for a healthcare app from ideation to launch.

●●● Hard

Hint: Define user need (doctors/patients) → data availability → model choice → regulatory (FDA?) → pilot → metrics → phased rollout.

Q: How do you prioritise between improving model accuracy vs reducing latency?

●● Medium

Hint: Depends on use case: fraud detection needs both; recommendation can trade accuracy for speed. Use user research + A/B data.

Q: What is a cold-start problem in AI products? How do you solve it?

●● Medium

Hint: New user/item has no history. Solutions: content-based filtering, demographic-based rules, onboarding questionnaire, popular items.

Q: How do you handle an AI model that is biased against a demographic group?

●●● Hard

Hint: Audit data + outputs by subgroup. Apply fairness constraints. Re-collect representative data. A/B test with fairness-aware model.

Q: Product manager says 'we need AI to fix retention.' How do you respond?

●●● Hard

Hint: Push back on 'AI as solution.' First understand WHY users churn. AI may help via personalisation, churn prediction, but root cause first.

Q: Walk me through how you would build a roadmap for an LLM-powered writing assistant.

●● Medium

Hint: Phase 1: basic autocomplete. Phase 2: tone/style. Phase 3: multi-document context. Prioritise by user pain, effort, and competitive moat.

SECTION 8 — Data Engineering, SQL & Databases

AI engineers spend more time wrangling data than writing models. Strong SQL, understanding of data pipelines, and knowledge of distributed systems are essential for any AI role.

Topic	Key Concepts
SQL	JOINS, window functions (ROW_NUMBER, LAG, LEAD), CTEs, GROUP BY, subqueries, indexing
Data Pipelines	ETL vs ELT, Apache Spark, Kafka for streaming, Flink, Airflow orchestration
Storage	Data lake (S3, GCS), data warehouse (BigQuery, Snowflake, Redshift), lakehouse (Delta Lake)
Vector Databases	Approximate nearest neighbour (FAISS, HNSW), Pinecone, Weaviate, Chroma, pgvector
NoSQL	MongoDB (document), Redis (cache/KV), Cassandra (wide-column), Neo4j (graph)
Data Quality	Great Expectations, dbt tests, schema validation, anomaly detection in pipelines

Interview Questions — Data Engineering & SQL

Q: Write a SQL query to find the top 3 products by revenue for each category in the last 30 days.

●● Medium

Hint: Use ROW_NUMBER() OVER (PARTITION BY category ORDER BY revenue DESC) in a CTE, then filter rank <= 3.

Q: Explain the difference between a data lake, data warehouse, and data lakehouse.

● Easy

Hint: Lake: raw data, cheap storage. Warehouse: structured, fast query. Lakehouse: combines both — raw storage + ACID transactions (Delta Lake).

Q: How would you design a data pipeline for training a recommendation model daily?

●●● Hard

Hint: Kafka → Spark Streaming → feature engineering → Delta Lake → Airflow trigger → training → MLflow → model registry.

Q: What is indexing in databases? How does it affect ML training pipelines?

●● Medium

Hint: Index speeds up reads (B-tree, hash). In ML: index on user_id, item_id, timestamp for fast feature queries. Trade: slower writes.

Q: How does FAISS work for vector similarity search?

●●● Hard

Hint: FAISS uses HNSW or IVF+PQ indexes for approximate nearest neighbour. Trades recall for speed. Essential for large embedding spaces.

SECTION 9 — Coding Round Patterns & Preparation

Most AI companies include a DSA coding round. AI-specific companies also include ML implementation tasks — coding a model from scratch in NumPy or PyTorch. Both require structured preparation.

Pattern	Key Technique	Common Problems
Sliding Window	Two pointers on array/string	Max subarray, longest substring
Binary Search	Divide sorted search space	Search rotated array, nth root
Dynamic Programming	Memoisation / tabulation	Knapsack, LCS, coin change
Graph BFS/DFS	Queue (BFS), recursion (DFS)	Shortest path, connected components
Heap / Priority Queue	heapq in Python	Kth largest, merge K sorted lists
Tree Traversal	Inorder, preorder, postorder	BST validation, lowest common ancestor
Backtracking	Recursive choices with pruning	N-queens, permutations, Sudoku
Matrix / 2D DP	State defined on grid cell	Unique paths, word search

ML Implementation Questions (Code from Scratch)

- Implement linear regression with gradient descent in pure NumPy.
- Implement K-Means clustering from scratch with random initialisation and convergence check.
- Implement a simple neural network forward + backward pass in NumPy.
- Implement softmax, cross-entropy loss, and their gradients.
- Implement cosine similarity and use it for nearest neighbour search.
- Implement a custom Dataset and DataLoader class in PyTorch.
- Implement attention mechanism: Q, K, V matrices, scaled dot-product, softmax.
- Implement BLEU score for evaluating text generation outputs.

Interview Tips for Coding Rounds

DO THESE

- ✓ Think aloud — explain your approach before coding
- ✓ Start with brute force, then optimise
- ✓ Clarify edge cases before starting
- ✓ Write clean variable names — readability matters
- ✓ Test with a small example on paper/whiteboard
- ✓ State time and space complexity at the end

AVOID THESE

- ✗ Jump straight to coding without a plan
- ✗ Use variable names like a, b, x, y throughout
- ✗ Forget to handle empty input / null cases
- ✗ Ignore the interviewer's hints completely
- ✗ Submit without tracing through the example
- ✗ Panic and erase everything — iterate instead

SECTION 10 — Responsible AI, Ethics & Current Trends

Top AI companies ask about responsible AI — expect at least 1-2 questions on fairness, bias, safety, and your awareness of cutting-edge research. Stay updated on ArXiv and major AI labs.

Topic	Key Points to Cover
Algorithmic Bias	Sources (data, labelling, feedback loops), mitigation, fairness metrics
Fairness Metrics	Demographic parity, equalised odds, calibration — know trade-offs between them
Explainability (XAI)	LIME (local approximation), SHAP (Shapley values), attention weights, saliency maps
AI Safety	Alignment problem, reward hacking, specification gaming, RLHF limitations
Privacy	Differential privacy, federated learning, data anonymisation, GDPR compliance
Environmental Impact	Carbon cost of training large models, efficient architectures, green AI
Hallucinations & Trust	Grounding, citations, uncertainty quantification, calibration
Current Trends	Multimodal LLMs, AI agents, foundation model fine-tuning, AI regulation

Interview Questions — Responsible AI

Q: How do you identify and mitigate bias in a hiring AI system?

●●● Hard

Hint: Audit predictions by gender/race/age. Measure equalised odds. Remove protected attributes (but beware proxy features). Diverse training data.

Q: Explain SHAP values. How do they differ from feature importances in Random Forest?

●●● Hard

Hint: SHAP uses Shapley theory — fairly distributes prediction credit among features. RF importances are mean impurity decrease, not causal.

Q: What is federated learning? What are its limitations?

●● Medium

Hint: Train on decentralised data without sharing raw data — model gradients only. Limitations: communication cost, non-IID data, gradient inversion attacks.

Q: How would you handle a model that performs well on average but poorly for a minority subgroup?

●● Medium

Hint: Evaluate disaggregated metrics. Re-sample or re-weight underrepresented group. Apply fairness constraints. Consider separate models.

SECTION 11 — HR & Behavioural Interview Questions

The HR round evaluates your soft skills, cultural fit, motivation, and how you handle real workplace situations. Use the **STAR Method** for all behavioural questions: **S**ituation → **T**ask → **A**ction → **R**esult.

S — Situation	Set the context. What was the situation or challenge? Be concise.
T — Task	What was YOUR specific responsibility or goal in this situation?
A — Action	What specific steps did YOU take? Use 'I', not 'we'. Be detailed.
R — Result	What was the measurable outcome? Numbers are powerful. What did you learn?

About Yourself

Q: Tell me about yourself.

Strategy: Structure: current role/study → key skills/achievements → why you're here. Keep it under 2 minutes. End with why this role excites you.

Q: What are your greatest strengths?

Strategy: Pick 2 strengths directly relevant to AI roles (e.g., problem-solving, learning agility). Back each with a brief example.

Q: What is your biggest weakness?

Strategy: Choose a REAL weakness you're actively improving. Never say 'I work too hard.' Example: public speaking → joined Toastmasters.

Q: Where do you see yourself in 5 years?

Strategy: Show ambition aligned with the company's direction. AI lead, staff engineer, or building AI products. Show you've thought about growth.

Motivation & Fit

Q: Why do you want to work in AI / this company?

Strategy: Genuine answer: specific products/research you admire, mission alignment, team reputation. Do your homework on the company's AI work.

Q: Why are you leaving your current job? (Experienced)

Strategy: Stay positive. Focus on growth opportunities, new challenges, company's AI ambitions — never badmouth your current employer.

Q: What do you know about our AI products/research?

Strategy: Do your homework! Mention specific papers, products, or blog posts. Show genuine curiosity, not flattery.

Q: How do you stay updated with AI developments?

Strategy: ArXiv, Papers With Code, AI newsletters (The Batch, Import AI), Twitter/X AI community, Hugging Face, reading actual papers.

Behavioural / STAR

Q: Tell me about a challenging ML project and how you handled it.

Strategy: Use STAR. Describe data/model challenge, your specific debugging/solution steps, and quantifiable improvement achieved.

Q: Describe a time you disagreed with your team on a technical decision.

Strategy: Show you raised concerns with data/reasoning, listened to others, and either changed your mind or reached a compromise. No 'I won' stories.

Q: Tell me about a time you failed. What did you learn?

Strategy: Pick a real failure. Show self-awareness, accountability, and concrete lesson applied afterward. Avoid blaming others.

Q: Describe a time you learned a new technology quickly to solve a problem.

Strategy: Great answer for AI roles: show learning agility, resourcefulness, and translating new knowledge into results fast.

Q: Tell me about a time you led a project or initiative.

Strategy: Even without a formal title, show leadership: setting direction, aligning teammates, unblocking obstacles, delivering results.

Q: How do you handle working under pressure or tight deadlines?

Strategy: Give a specific example. Show prioritisation, communication with stakeholders, breaking work into sprints, and maintaining quality.

AI-Specific HR Questions

Q: What is your opinion on the impact of AI on jobs?

Strategy: Balanced, thoughtful answer. AI automates routine tasks, creates new roles. Emphasise responsibility of AI builders to think about societal impact.

Q: How do you approach ethics in your AI work?

Strategy: Show you think about bias, fairness, and privacy by default — not as an afterthought. Mention specific practices (audits, fairness metrics).

Q: Describe your experience with large language models.

Strategy: Be specific: which models, what tasks, fine-tuning or prompt engineering, production deployment, challenges faced (cost, latency, hallucinations).

Q: How do you communicate complex AI concepts to non-technical stakeholders?

Strategy: Use analogies, avoid jargon, focus on business impact not accuracy numbers. Show empathy for the audience's level of understanding.

Salary & Logistics

Q: What are your salary expectations?

Strategy: Research market rate (Glassdoor, LinkedIn, levels.fyi). Give a range, not a fixed number. Say 'open to discussion based on total comp.'

Q: Do you have other offers / are you in other interview processes?

Strategy: Be honest but don't reveal specifics. 'I'm exploring a few opportunities' is fine. Creates healthy urgency without pressure.

Q: When can you join? / What is your notice period?

Strategy: Give an honest answer. If negotiating for earlier join, note it: 'Standard is 60 days but I can negotiate buyout or a mid-handover.'

Q: Do you have any questions for us?

Strategy: ALWAYS have 3-5 thoughtful questions. Ask about the AI tech stack, team structure, biggest challenges, how success is measured in 90 days.

Smart Questions to Ask YOUR Interviewer

- What does your AI infrastructure look like — what tools and platforms does the team use day to day?
- What is the biggest technical challenge the AI team is facing in the next 6 months?
- How does the team measure model success — are you primarily metric-driven or user-research-driven?
- How much autonomy do engineers/PMs have in choosing model architectures or product direction?
- What does the onboarding process look like for a new AI engineer/product developer?
- What opportunities exist for publishing research or contributing to open-source projects?
- How do you balance moving fast with ensuring AI safety and responsible deployment?

Recommended Study Plan & Final Tips

Week	Focus Area	Daily Target
Week 1	Python fundamentals + Math/Stats + SQL	2 hours study + 2 LeetCode problems
Week 2	Machine Learning algorithms (core concepts + questions)	Read 1 topic + solve 5 questions
Week 3	Deep Learning + NLP + Transformers	1 architecture/day + implement in code
Week 4	LLMs + RAG + Fine-tuning + GenAI	Build a mini project with LLMs
Week 5	MLOps + System Design (mock design sessions)	1 full system design per day
Week 6	HR prep + Mock interviews + Revise weak areas	2 mock interviews + STAR stories

Final Preparation Tips

- Practice on platforms: LeetCode (DSA), Kaggle (ML), Hugging Face (LLMs), StrataScratch (SQL).
- Do at least 3 mock interviews with a peer — speaking aloud is very different from knowing the answer.
- Build at least 1 end-to-end AI project you can discuss for 15-20 minutes in depth.
- Read company engineering blogs (Google Brain, Meta AI, OpenAI, Anthropic) before interviews.
- Know your own resume deeply — every project, every metric, every technology should be defensible.
- Sleep well the night before. Calm confidence outperforms frantic last-minute cramming.

The best AI engineers are not those who memorise algorithms — they are those who understand the 'why' behind every decision. Build things. Break things. Learn deeply. Go get that offer!

Best of luck on your AI journey!